

Forms [Part-2]

Please before starting, go and read twig docs + most importantly, read # How to Work with Form Themes docs page

The conclusion of what will happen is this: you create a theme file under `src/templates/form/themes` and apply this theme file by adding its path to twig.yaml config file

what does this theme file contain is what we will see and how this file affect our forms in the app.

we have already said that `{{ form_row(form.email) }}` displays all row parts including: label, error message, widget and help, but you can display each one using their helpers like:

```
<div class="form-control">
  <i class="fa fa-calendar"></i> {{ form_label(form.dueDate) }}
  {{ form_widget(form.dueDate) }}
  <small>{{ form_help(form.dueDate) }}</small>
  <div class="form-error">
    {{ form_errors(form.dueDate) }}
  </div>
</div>
```

In symfony to customize a field in a way that whenever we have this field with name for example == email and it is text input; then some template or block (within theme file) will be used to globalize for example the look for each email input in the app. To do so, you need first choose a type to customize and then:

- create a file : `src/templates/form/themes/generic_fields.html.twig`

[Form Fragment Naming](https://symfony.com/doc/current/form/form_themes.html#form-fragment-naming "Permalink to this headline")

The naming of form fragments and block within this file varies depending on your needs:

- If you want to customize **all fields of the same type** (e.g. all `<textarea>`) use the ``field-type_field-part`` pattern (e.g. ``textarea_widget``).
- If you want to customize **only one specific field** (e.g. the `<textarea>` used for the ``description`` field of the form that edits products) use the ``_field-id_field-part`` pattern (e.g. ``_product_description_widget``).

In both cases, the ``field-part`` can be any of these valid form field parts:

Here is an example of email in bootstrap

in twig file, in order to customize the way a field is rendered, symfony tells you to use **blocks** and use some naming rules like beginning block with name of widget (email in this case) and add underscore row like following:

```
{{ block_email_row }}

<div class="mb-3">
  <label for="email" class="form-label">Email address</label>
  <input type="email" class="form-control" id="email"
    aria-describedby="emailHelp" placeholder="name@example.com">
  <div id="emailHelp" class="form-text">
```

```

        We'll never share your email with anyone else.</div>
    </div>

    {{ endblock email_row }}

```

Now how we can tell symfony to use this template in all fields email in my application

To do so, we use a config file called twig.yaml where you give it the path to the theme file as form_themes (path started from templates):

```

twig:
    file_name_pattern: "*.twig"
    form_themes:
        - 'form/themes/generic_fields.html.twig'

```

Here : `{{ block email\_row }}` we used `field-type\_field-part`, so any form control with type email will get this theme

Now the block that we have in theme is static, and when we have an email input in our FormType, the content of the block will be injected but symfony will not put the values like name value and the necessary informations to the block, so we need to make the block somehow dynamic.

inside the block, you have access to form variables (see docs), one of them is form

form variable (has the form view in which you are in), and with that, now we can use helpers and pass this form to them in order to make it dynamic:

```

    {{ block email_row }}

        <div class="mb-3">
            {{ form_label(form) }}
            {{ form_widget(form) }}
            {{ form_help(form) }}
            ....
            # use dump to print all variables inside the block
            {{ dump() }}
        </div>

    {{ endblock email_row }}

```

Now, if we use form_row for email, the above block will be used so we now have the ability to customize it and decide how it will look

Do in controller:

```

$form = $this->createForm(type: ContactUsType::class, data: [
    "emailAddress" => "mouad@gmail.com"
]);

```

Now inside blocks, you have access to form variables that help you with everything you need about the formtype like value, form, label, help... (see docs: symfony form variables reference)

Also look at the dump() in the previous block twig code above, it helps you dump everything related to the form type and see all values of each variable.

- let's add help to email field in formtype class

```

public function buildForm
    (FormBuilderInterface $builder, array $options): void
{
    $builder->add('email', EmailType::class, options: [

```

```

        "help" => "please enter your email vcorrect"
    });
}

```

Now if you refresh your page and dump inside block, you will see help filled with the value you gave in buildForm, and so you can access it inside your partial block and display it

Please notice that all helpers like form_widget, form_help.. take an argument which is form which you can get by using form variable as shown in the previous twig snippet

Sometimes you want to customize all helps no matter the type of widget, because email, input textarea... extend from something called form, now to customize something in general regardless its widget type, you name it form_help like so

```

{{ block form_help }}
    <p class="text-muted small">{{ help }}</p>
{{ endblock form_help }}

```

Here we use form variable called **help** to get help option passed earlier from buildForm

Please note:

Do we need to handle all cases and do all this work for all controls and form parts? of course no, since Symfony already gave you solution to this in case you are using bootstrap. Take a look at docs: Symfony built-in form themes.

Symfony forms use by default the `form_div_layout.html.twig` to help you use predefined theme but you can use your own or some provided themes that symfony gave you (see form themes in docs ex: `[foundation_6_layout.html.twig](https://github.com/symfony/symfony/blob/master/src/Symfony/Bridge/Twig/Resources/views/Form/foundation_6_layout.html.twig)`)

Now in order to use it, we need to add it into `twig.yaml`, and you don't have to copy its path, since it is provided by symfony you just put its name in the `form_themes` array directly like so:

```
twig:
  file_name_pattern: "*.twig"
  form_themes:
    - 'form/themes/generic_fields.html.twig'
    - 'foundation_6_layout.html.twig'
```

IMPORTANT: the order is important; the last theme override the previous one, so the `foundation_6_layout`, will override your own block (`generic_fields`)

Up until now, we have a block called `form_help`, try to go to bootstrap file and look for `form_help` and see how they managed to implement it

If you want your own theme to take precedence over bootstrap one, place your file path at the end of array in `twig.yaml` `form_themes`

===

Symfony recommends that for the form to be more re-usable, try to avoid defining classes in `buildForm` function like we did previously:

```
$builder
    ->add('email', EmailType::class, [
        "attr" => [
            "class" => "form-control" <=== discouraged
        ]
    ])
;
```

This is because you need to make classes dynamic when using the same view in multiple places, so the definition of classes should be done in the moment when you call it and render it by using the second argument of `form_row` for example: (in a twig page)

```
{{ form_row(contact_us.message, { attr: { class: "my-own-class" } }) }}
```

don't worry, symfony merge your classes with bootstrap classes and not use only the classes defined in the above snippet. (the following is from `bootstrap_5_layout.html.twig` showing how it is done [merge])

```
{%- block widget_attributes -%}
{%- if not valid %}
    {% set attr = attr|merge({class: (attr.class|default('')
        ~ 'is-invalid')|trim}) %}
{% endif -%}
{{ parent() }}
{%- endblock widget_attributes -%}
```

The merge is applied to the classes that we have in the **FormType class**, with the **classes defined in the block** with the **classes of bootstrap layout**.

so to use classes while using the form row, we can use the second argument like so

```
{{ form_row(form.name, {'label': 'foo'}) }}
```

So the second argument is nothing but options you use in builder to build form

```
$builder->add('email', EmailType::class, options: [
    "label" => "Email address"
]);
```

Now you can remove the label from builder->add to make your form very reusable and pass it when displaying it when calling form_row(). Another example:

```
{{ form_row(form.name, {'attr': { class: "sudowr-class" }}) }}
```

To get a block inside another block, you can do: block(name_of_included_block)

```
-> `{{ block(widget_attributes) }}`
```

When configuring attributes, there are some of them that are configurable directly in options arg in

createFormBuilder ### [Creating Forms in

Controllers](<https://symfony.com/doc/current/forms.html#creating-forms-in-controllers> "Permalink to this headline"), and some of them are accessible through attr, so to know, you can go to docs : ##

[Supported Field

Types](<https://symfony.com/doc/current/reference/forms/types.html#supported-field-types> "Permalink to this headline") and see if the attribute you want to handle is there if so use it directly if not put it inside attr

In options, there is label, and there is label_html where label_html take the content and interpret it to html where if you just specify label alone, then the content as it is will be shown (no parsing is there), so adding label_html control the appearance by parsing the label content to html

Please note that textarea is text_type just like inputs, because all controls in symfony have one parent type is TextType and the way it has this parent not by extending it, but they all have a method called getParent() return TextType::class

Apply style to one specific field

Sometimes, applying field style to all widgets will cause all widgets with that type to take the new shape which is not the case most of the time

Now we need to tell symfony, instead of applying this widget or style to all text inputs, we can tell it to apply it to one specific field: see docs (Fragment Naming for Individual Fields)

We already talked about the two ways of fragmenting we used previously `field-type\_field-part` but in this case to specify one specific element to customize its theme we need to use `\_field-id\_field-part`

-> `_contact_us_email_row`

```
{{ block _contact_us_email_row }}  
    <div class="mb-3">  
        <label for="exampleInputEmail1" class="form-label">Email address</label>  
        <input type="email" class="form-control" id="exampleInputEmail1"  
            aria-describedby="emailHelp" placeholder="name@example.com">  
        <div id="emailHelp" class="form-text">  
            We'll never share your email with anyone else.</div>  
        </div>  
  
    {{ endblock _contact_us_email_row }}
```

Here `_contact_us_email_row` is generated by taking `_` then add the snake case of our form class, and `_` and email which is the field name then `_` then part you want to customize (row, widget,...)

sometimes you need to do an if to check if for example help content is not equal to empty string, if yes, then show a span with help content

```
{% block form_help %}  
    {% if help|trim is not same as('') %}  
        <small class="form-text text-muted">{{ help }}</small>  
    {% endif %}  
{% endblock form_help %}
```

(from docs: [Disabling Global Themes for Single Forms](https://symfony.com/doc/current/form/form_themes.html#disabling-global-themes-for-single-forms) "Permalink to this headline")

Sometimes you may need to apply a theme only to some specific form. You can do that with the `[form_theme Twig tag]`(https://symfony.com/doc/current/reference/twig_reference.html#reference-twig-tag-form-theme) inside the twig file where you have the form displayed:

Take a look at the code below and then come back: the `form_theme` take the form you want to work with and it may take another argument: this argument is `_self` if your theme is defined in the same file, but if your theme is present in another file, you need to give it the path where the theme is.

```
{# this form theme will be applied only to the form of this template #}  
{% form_theme contact_us 'form/themes/contact_us/custom_fields.html.twig' %}  
  
{{ form_start(form) }}  
    {# ... #}  
{{ form_end(form) }}
```

you can also achieve the same thing by defining the widget style in the same place as the form like you don't want to create a dedicated template to customize just one field; to do so:

```
{% form_theme form _self %}
```

-> Then place your block at the end of twig file to allow the file to use it to customize the form inside it

You can also use multiple themes using **with** like so:

```
{% form_theme contact_us with ['path-to-theme-1', 'path-to-theme-2', '...'] %}
```

You can customize the form in total, you can use `block_prefix` and `block_name` to handle it

Let say we have a formtype like this one

```
class ContactUsType extends AbstractType {
    public function buildForm(FormBuilderInterface, array): void {}
}
```

If we don't change neither prefix nor name of block here what will have then Symfony automatically generates a **block prefix** from the class name. `ContactUsType` → `contact_us`

(Type name → remove ``Type`` → `snake_case`), Twig then use these blocks when you render:

`{{ form(form) }}` Twig will look for: ``contact_us_widget`` and ``contact_us_row``

If they don't exist, it falls back to: ``form_widget form_row``

If inside the form you have: ``$builder->add('email', TextType::class);`` Twig will try blocks in this order (simplified):

``_contact_us_email_widget``

``email_widget``

``text_widget``

``form_widget``

From most specific → most generic.

To conclude: Without changing anything:

- Form block prefix = ``contact_us``
- Twig can override whole form using: ``{% block contact_us_widget %}``
- Each field gets automatic naming based on: form prefix, field name, field type

Customizing block_prefix and block_name:

Given:

```
`class ContactUsType extends AbstractType`
```

Symfony automatically generates: `block\_prefix = contact\_us`

This affects: HTML `name` attribute, HTML `id` and Twig block hierarchy

Example HTML: `

How to remove prefix completely

If you return empty string: `public function getBlockPrefix(): string { return ""; }`

Result: `

No more `contact\_us[...]`. (This is often used for API forms.)

Changing prefix per field (not whole form)

Inside `buildForm()`:

```
$builder->add('email', TextType::class, [
    'block_name' => 'custom_email',
]);
```

This changes only the Twig block naming logic for that field.

Important Clarification

There are two different things:

1. Form prefix (controlled by `getBlockPrefix()`)
→ affects HTML + Twig hierarchy
2. `block\_prefix` / `block\_name` (options)
→ affect only Twig block resolution

In conclusion, to show these customizations that applied to the whole form in totality, you should render the whole form like:

```
{{ form_start(form) }}
    {{ form_row(form) }}
    <button type="submit">Send</button>
{{ form_end(form) }}
```

How block_prefix and block_name works

Assume: `class ContactUsType extends AbstractType`

And inside: `\$builder->add('email', TextType::class);`

Twig builds a hierarchy like this:

`\_contact\_us\_email\_widget` ← field-specific`

`contact\_us\_widget` ← form-level`

`text\_widget` ← field type`

`form\_widget` ← generic`

What `block\_name` changes

```
$builder->add('email', TextType::class, [
    'block_name' => 'custom_email',
]);
```

Now the first block becomes: `\_custom\_email\_widget`

Instead of: `\_contact\_us\_email\_widget`

It changes the **field-specific name only**. It does NOT affect `text\_widget`.

What `block\_prefix` changes

```
$builder->add('email', TextType::class, [
    'block_prefix' => 'special',
]);
```

Now the hierarchy becomes:

`\_contact\_us\_email\_widget`

`contact\_us\_widget`

`special\_widget`

`form\_widget`

Instead of: `text\_widget` it replaces the **field type block** (`text\_widget`).

It does NOT change `\_contact\_us\_email\_widget`.